

LLD – Paper Submission Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Paper Submission module of OpenScholar. It covers API contracts, data model usage, service logic, storage integration, validation, and edge cases. The module supports draft creation, submission, versioning, multi-author management (registered and external), and metadata handling.

2. Scope

The Paper module is responsible for:

- Create paper (draft)
- Update paper metadata
- Upload PDF file ($\leq 50\text{MB}$)
- Create new versions
- Manage authors (registered + external)
- Maintain author ordering per version
- Submit paper for moderation
- Retrieve paper details (including versions)

Out of Scope:

- Admin moderation decisions (covered in Moderation module)
 - Search (covered in Search module)
-

3. Data Model (Relevant Tables)

Tables used:

- papers
- paper_versions
- authors
- paper_authors
- paper_tags
- users

Key constraints:

- UNIQUE (paper_id, version_number) on paper_versions
- PRIMARY KEY (paper_version_id, author_id) on paper_authors

- ON DELETE CASCADE from papers → paper_versions → paper_authors
-

4. API Contracts

4.1 Create Paper (Draft)

Endpoint: POST /api/papers

Request (multipart/form-data):

- file: PDF (optional at draft stage)
- title: string (optional for draft)
- abstract: string (optional for draft)
- keywords: string[] (optional for draft)
- categoryId: number (optional for draft)
- tagIds: number[] (optional)
- authors: array (optional for draft)
- [{ "name": "Author Name", "userId": "uuid | null", "order": number }]

Validation Rules:

- If file is provided, it must be PDF and $\leq 50\text{MB}$

Response: { "paperId": "uuid", "status": "draft" }

4.2 Update Draft Paper

Endpoint: PUT /api/papers/{paperId}

Request (multipart/form-data):

- file (optional)
- title (optional)
- abstract (optional)
- keywords (optional)
- categoryId (optional)
- tagIds (optional)
- authors (optional)

Response: { "message": "Draft updated successfully" }

4.3 Submit Paper for Review

Endpoint: POST /api/papers/{paperId}/submit

Validation Rules:

- title, abstract, keywords required
- at least one author required
- PDF file required

Response: { "message": "Paper submitted for review", "status": "pending" }

4.4 Create New Version

Endpoint: POST /api/papers/{paperId}/versions

Request (multipart/form-data):

- file: PDF (required)
- title: string
- abstract: string
- keywords: string[]
- authors: array

Response: { "versionId": "uuid", "versionNumber": 2 }

4.5 Get Paper Details

Endpoint: GET /api/papers/{paperId}

Response: { "id": "uuid", "status": "approved", "versions": [{ "versionNumber": 1, "title": "...", "pdfUrl": "...", "authors": [] }] }

5. Storage Design

File Upload Flow

1. Receive file via API (multipart)
2. Validate file type and size
3. Upload to MinIO (S3-compatible)
4. Store returned URL in paper_versions.pdf_url

File Naming Strategy:

- papers/{paperId}/v{versionNumber}.pdf
-

6. Service Logic

6.1 createPaperDraft()

Steps: 1. Create paper record with status = 'draft' 2. If file provided:

- Upload file
 - Create initial version (version_number = 1)
 - Save metadata (if provided)
 - Insert authors (if provided)
 - Insert tags
-

6.2 updateDraft()

Steps: 1. Validate ownership (created_by user) 2. Update metadata fields 3. If file provided:

- Upload new file
 - Update current version
 - Update authors (replace existing mapping)
 - Update tags
-

6.3 submitPaper()

Steps: 1. Validate required fields (title, abstract, keywords, authors, file) 2. Update paper status to 'pending'

6.4 createNewVersion()

Steps: 1. Fetch latest version_number 2. Increment version_number 3. Upload new PDF 4. Create new paper_versions row 5. Insert authors for new version 6. Mark previous versions as not published

6.5 getPaperDetails()

Steps: 1. Fetch paper by id (exclude deleted) 2. Fetch all versions 3. Fetch authors per version 4. Return structured response

7. Validation Rules

- File must be PDF
- File size $\leq$ 50MB
- title, abstract required on submit

- keywords array must not be empty on submit
 - authors must have unique order values
 - author_order must start from 1 and be continuous
-

8. Security Considerations

- Only owner can update draft
 - Only owner can submit paper
 - Validate user identity via JWT middleware
 - Prevent unauthorized access to drafts
-

9. Edge Cases

- Duplicate author order
 - Missing required fields on submit
 - Upload failure (rollback DB if needed)
 - Updating non-existent paper
 - Unauthorized update attempt
-

10. Prisma Model Mapping (Relevant)

```
model Paper { id String @id @default(uuid()) status String categoryId Int? createdBy String isDeleted Boolean @default(false) createdAt DateTime @default(now()) }
```

```
model PaperVersion { id String @id @default(uuid()) paperId String versionNumber Int title String abstract String keywords String[] pdfUrl String isPublished Boolean createdAt DateTime @default(now()) }
```

```
model Author { id String @id @default(uuid()) name String userId String? }
```

```
model PaperAuthor { paperVersionId String authorId String authorOrder Int @@id([paperVersionId, authorId]) }
```

11. Folder Structure

```
src/ app/api/papers/route.ts app/api/papers/[id]/route.ts app/api/papers/[id]/submit/route.ts app/api/papers/[id]/versions/route.ts
```

```
modules/paper/service.ts modules/paper/repository.ts
```

```
lib/storage.ts
```

12. Implementation Notes

- Use Prisma transactions for version creation + author insertion
 - Use streaming upload for large files (future improvement)
 - Keep API routes thin and delegate to service layer
 - Maintain consistent response format
-

13. Acceptance Criteria

- User can create draft paper
 - User can update draft
 - User can upload PDF ≤ 50 MB
 - User can add multiple authors with ordering
 - User can submit paper for review
 - User can create new versions
 - System stores files correctly in storage
 - System enforces validation rules
-

14. Future Enhancements

- Presigned URL upload (direct to S3)
 - File version diff tracking
 - Auto-save drafts
 - Plagiarism API integration
 - DOI assignment
-

End of Document